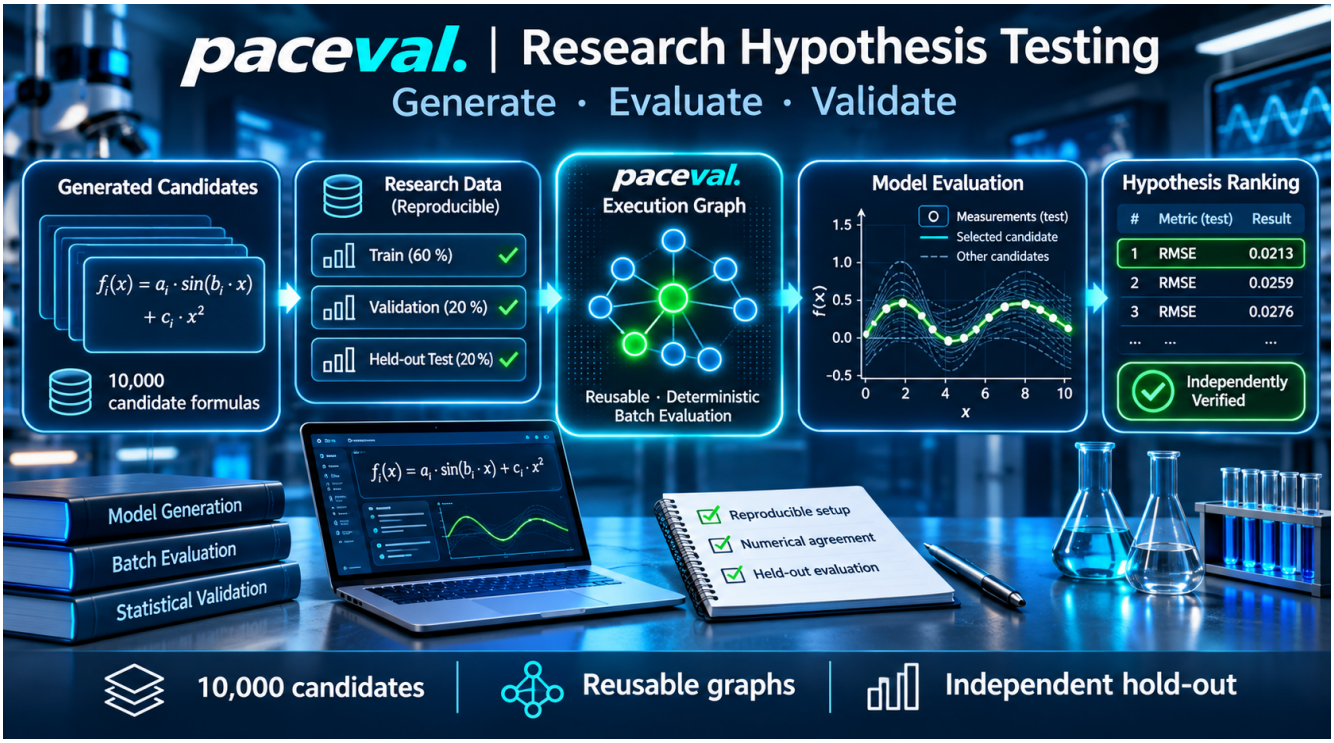
Batch Testing of Generated Research Hypotheses

A reproducible C++ feasibility demonstrator using paceval execution graphs

Scientific research report | 11 November 2026



Research question. Can a prepared mathematical execution layer evaluate thousands of automatically generated, explicitly specified research hypotheses consistently while keeping statistical model selection and independent validation separate?

Result. Yes, for the controlled synthetic model family studied here. The program evaluates 10,000 coefficient-bound hypotheses using eight reusable paceval graphs; checks numerical agreement against independent implementations; and selects one model using training and validation only before examining held-out and extrapolation partitions.

The known reference expression is deliberately included in the candidate set. Recovery is therefore a software feasibility result rather than a discovery of an unknown scientific law. No execution-speed comparison is asserted.

1. Scientific foundation and implementation boundary

Modern symbolic-regression and sparse-model-discovery systems generate interpretable mathematical hypotheses. Brunton et al. (2016) show how the search for sparse nonlinear governing equations can be separated from evaluating candidate functional forms. PySR / SymbolicRegression.jl likewise provides expression search and model-complexity selection. The present experiment isolates a distinct step: executing large numbers of proposed, explicit mathematical expressions.

The measured boundary is deliberate. The host program generates hypotheses, defines immutable data partitions, calculates mean-squared errors, selects a candidate and produces reproducible records. The paceval library creates reusable mathematical execution graphs and evaluates those graphs repeatedly with new input values and coefficients. No optimizer, stochastic generator, statistical ranking algorithm, or LLM is attributed to paceval.

1.1 Exact execution architecture

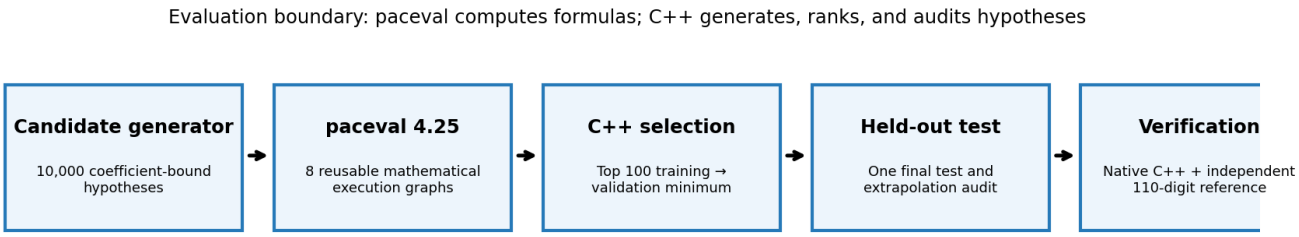


Figure 1. Actual tested architecture. Eight parameterized graph structures are shared by 10,000 coefficient-bound candidate hypotheses. The C++ host owns experiment design and selection.

1.2 Why shared graph structures matter

Each candidate is a distinct coefficient-bound mathematical hypothesis, written as a structural expression plus a uniquely specified vector of numerical coefficients. A computationally equivalent representation supplies the coefficients as variables to paceval. Consequently, 10,000 candidates do not require constructing 10,000 separate computation objects; the eight representative graph structures are constructed once and reused for all candidate/data combinations.

The method is particularly applicable to generated scientific hypotheses with recurring functional families. A future search method may produce entirely new expression trees; those would require additional graph creation after validating the expressions. This integration boundary is explicit rather than hidden.

The original proposal specifies 10,000 synthetic candidate expressions evaluated over fixed research partitions. This implementation instantiates that proposal using eight declared families with 10,000 unique coefficient bindings, and adds held-out plus extrapolation checks.

2. Mathematical models and candidate generation

The independently defined synthetic target is $f^*(x) = 1.4 \sin(1.8\,x) + 0.3\,x^2 + 0.12$. To create noisy observations, the C++ host adds a small deterministic oscillatory perturbation. The reference model is known by construction and injected as candidate 0. All quantities are dimensionless; no experimental materials or physical law are represented.

2.1 Exact expressions passed to paceval

Family	Function string used to create the paceval graph
0	a*sin(b*x)+c*x*x+d
1	a*cos(b*x)+c*x*x+d
2	a*tanh(b*x)+c*x*x+d
3	a*exp(-b*x*x)+c*x*x+d
4	a*sin(b*x)+c*x+d
5	a*cos(b*x)+c*x+d
6	a*sin(b*x)+c*x*x+d*cos(x)
7	a*ln(1+b*x*x)+c*x*x+d

The fixed variable list is x a b c d. Candidate coefficient vectors are generated with a fixed seed, 20261111, and remain unchanged across compiler and batch experiments. The logarithmic family uses positive b, keeping $1+b\,x^2$ positive on the specified data domain.

2.2 Scope of the candidate pool

10,000 different coefficient tuples form 10,000 distinct parameterized mathematical models. Their family membership cycles through the eight structures. The first model is planted exactly and disclosed in the source. Coefficients are not fitted by gradient descent or inferred by paceval; the experiment tests and ranks the generated candidates.

The full candidate catalog, including each explicit formula family, coefficient values and training errors, is published in examples/hypotheses.csv. This makes the experiment independently inspectable without knowing the initial random-generator state.

3. Statistical protocol: protecting the held-out test

The experiment uses four distinct, deterministic partitions. Their data points have separate row identifiers and shifted x grids. A synthetic-noise generator is defined in the C++ source and is held constant for every candidate.

Partition	Count	Domain	Use in the experiment
Training	120	x in [-2.4, 2.4]	Rank all 10,000 generated hypotheses
Validation	60	x in [-2.35, 2.35]	Choose one from the training shortlist
Held-out test	60	x in [-2.36, 2.36]	One final evaluation after selection
Extrapolation	48	x in [2.6, 3.8]	Separately examine out-of-domain behavior

3.1 Predefined selection procedure

• Compute training mean squared error (MSE) for all candidates using the prepared mathematical graphs.

• Rank by training error and keep exactly the best 100 candidates.

• Evaluate only these 100 candidates on the validation partition and choose the lowest validation MSE; resolve ties deterministically.

• Freeze the winner. Examine the held-out test and the separate extrapolation domain only after that decision.

This avoids selecting a winner by the reported final-test error. It does not by itself certify scientific truth or eliminate all possible selection bias in future research datasets.

3.2 Reproducibility controls

The program records the candidate identifier, formula family, coefficient vector, split membership, errors and exact selected outputs. The Python audit independently verifies the training shortlist and selection logic by reading the published CSV files. All candidate and data-generation decisions are held constant when the paceval batch size changes.

The full source of this pipeline is src/hypothesis_test.cpp, and the frozen dataset appears in examples/dataset.csv.

4. paceval C/C++ SDK implementation

The demonstrator includes the official paceval_main.h and paceval_main.cpp forwarding sources and links against a separately supplied compatible Linux runtime. It does not use a custom dlsym/dlopen API adapter, and no proprietary .so is bundled in the publication package.

4.1 Executed paceval API operations

API	Verified role
paceval_InitializeLibrary	Initialize the actual runtime once
paceval_CreateComputation	Construct eight graph families plus one TINC graph
paceval_dGetComputationResultExt	Execute candidate/sample batches; check per-sample error codes
paceval_dGetComputationResult	Read nominal result and TINC lower/upper bound
paceval_GetIsError	Verify handle status
paceval_DeleteComputation / paceval_FreeLibrary	Release each handle and the runtime

The batch API receives data in sample-major order: [x,a,b,c,d] for each sample. The five-input graph structure is unchanged while x and the bound coefficients vary. Errors or nonfinite outputs abort the experiment rather than silently poisoning model rankings.

4.2 Independent software responsibilities

• paceval: parse and prepare the mathematical expression, reuse its graph and compute numerical values for input batches.

• C++: generate candidates, create synthetic observations, compute MSE, rank models, enforce partition isolation and write data artifacts.

• Python/mpmath: independently re-evaluate candidate predictions and verify numerical agreement with 110 decimal digits.

4.3 Interval-enabled diagnostic graph

A ninth graph is created with the paceval TINC mode enabled. Twenty-four representative inputs are checked independently against the high-precision mathematical expression. These checks document observed containment on a finite sample; they do not establish universal bounds for every function, compiler or hardware combination.

5. Experimental observations with the original runtime

Observed evidence	Full study
Generated coefficient-bound hypotheses	10,000
Reusable mathematical family graphs	8 (+1 interval graph)
Native C++ numerical spot checks	10,818; zero violations
Independent high-precision candidate samples	144; zero tolerance violations
Independent high-precision selected-model predictions	288; zero tolerance violations
Pointwise TINC containment observations	24 of 24
Chosen candidate after train/validation selection	ID 0; family 0; the disclosed planted target

5.1 Selected hypothesis and final partitions

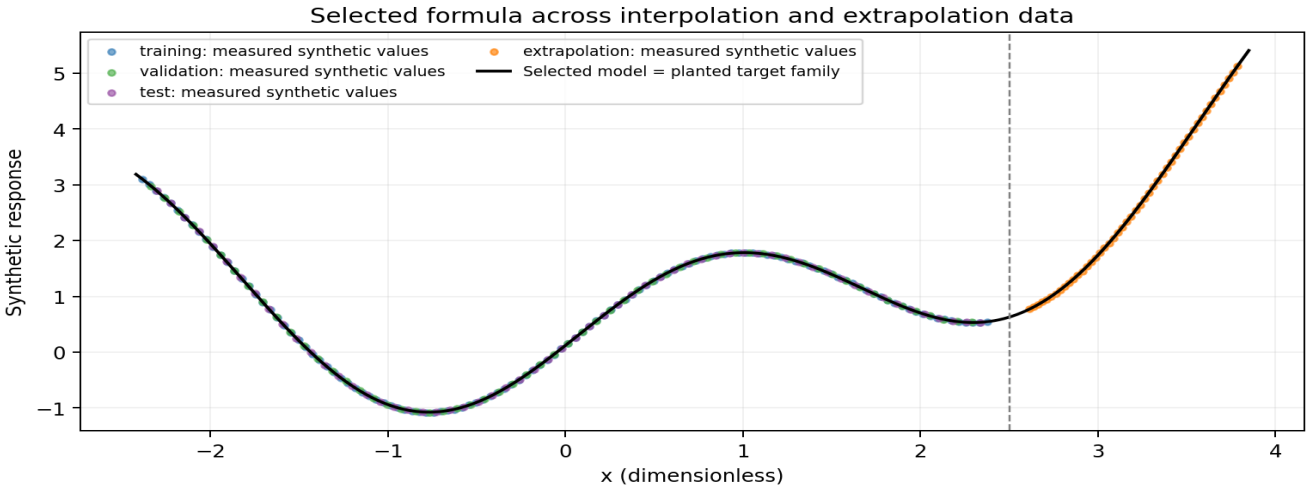


Figure 2. Actual synthetic observations from all four partitions and the selected analytical curve. The rightmost region is an extrapolation check, not a model-selection input.

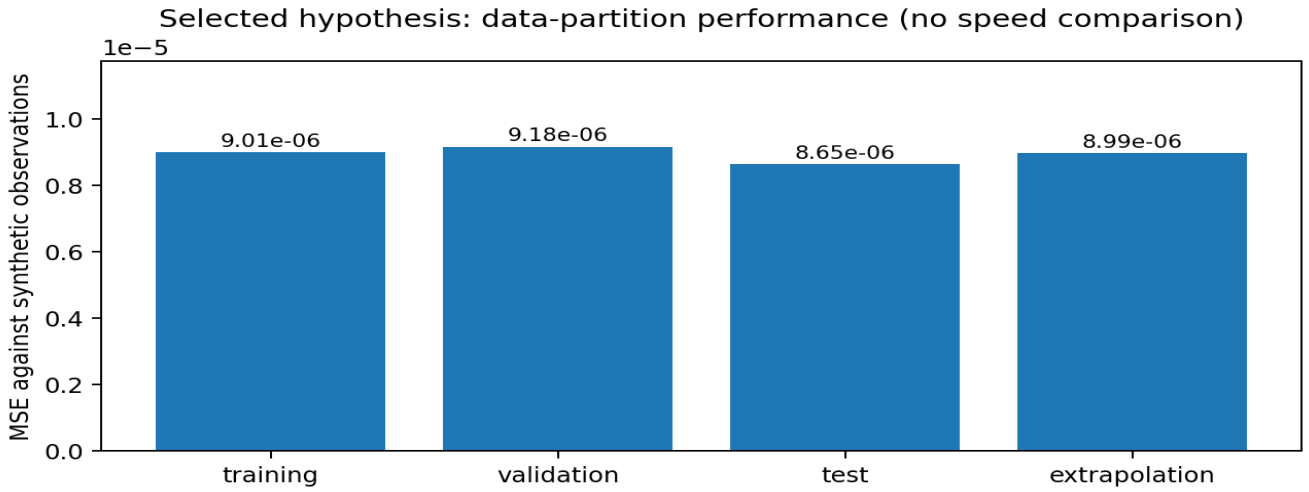


Figure 3. Mean squared error on each partition from the reproduced CSV outputs. The comparable MSE scale reflects the deliberately planted exact hypothesis and deterministic observation noise.

6. Candidate ranking and validation

All 10,000 candidates are scored on training data; the train-score distribution spans several orders of magnitude. Only the 100 best training candidates are evaluated on the validation set. In the controlled experiment the planted target is ranked and selected as the final formula.

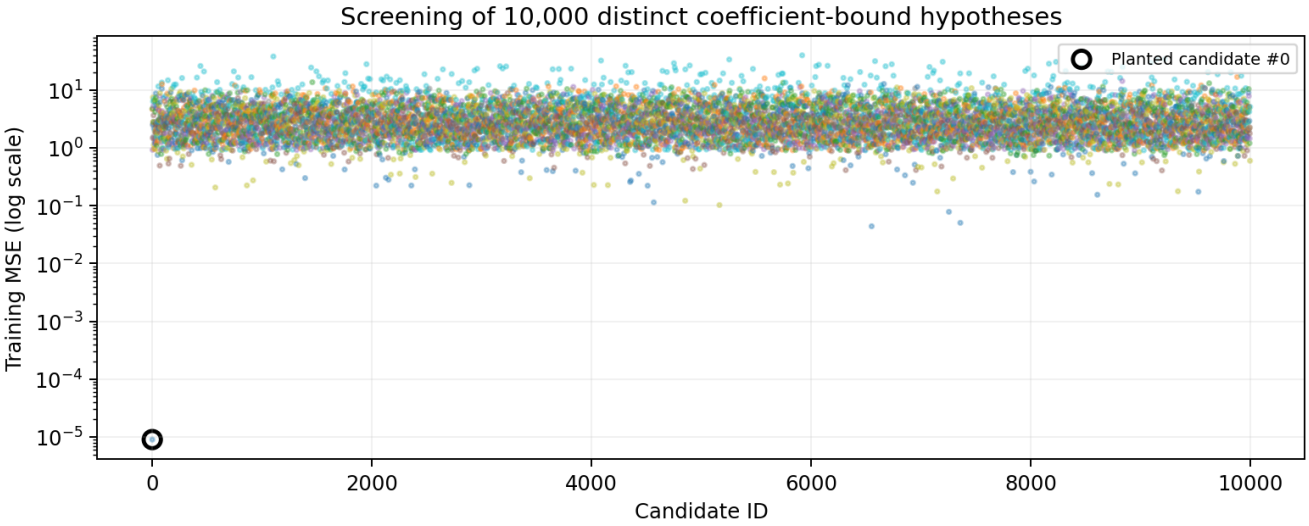


Figure 4. The complete training-screening landscape. Every point is an actual generated hypothesis evaluated through paceval.

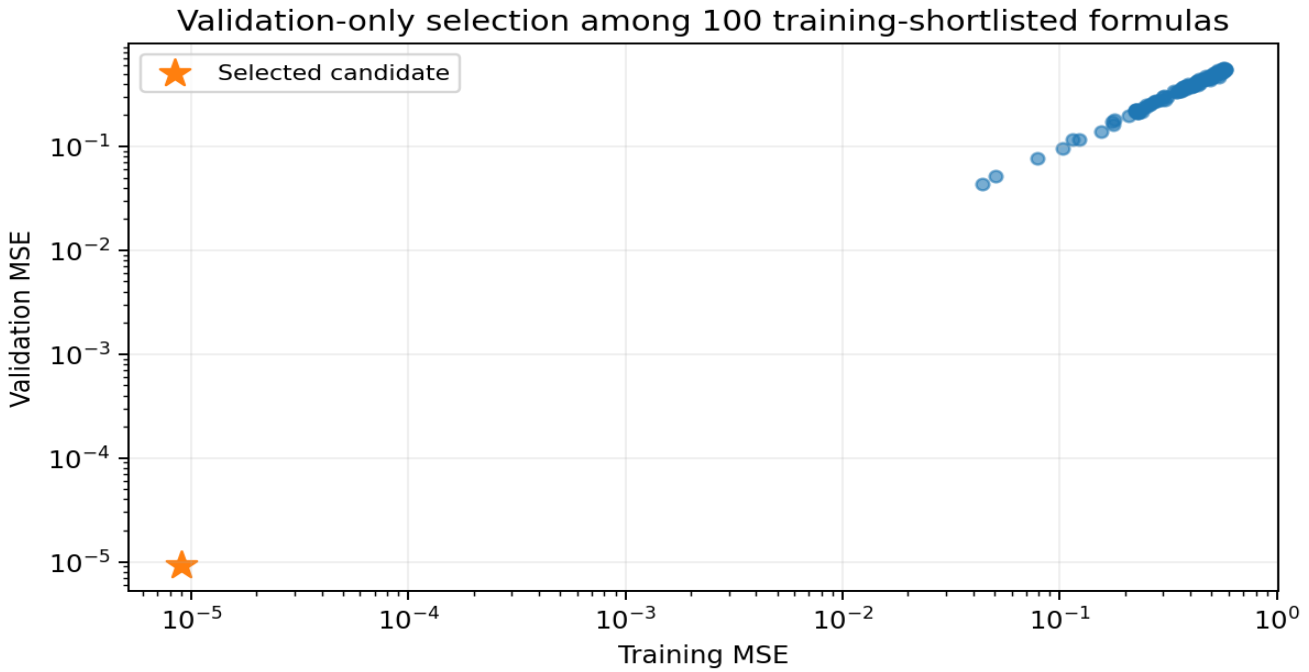


Figure 5. Training and validation losses for the 100 shortlisted candidates. Candidate choice depends on this validation result, not on held-out test error.

7. Independent numerical validity checks

The C++ executable implements all eight formulas independently for sampled numerical comparisons. A second audit uses mpmath with 110 decimal digits for 144 sampled candidate outputs and all 288 selected-model predictions. The maximum absolute observed deviation to this independent reference is approximately 9.3370575933743e-16.

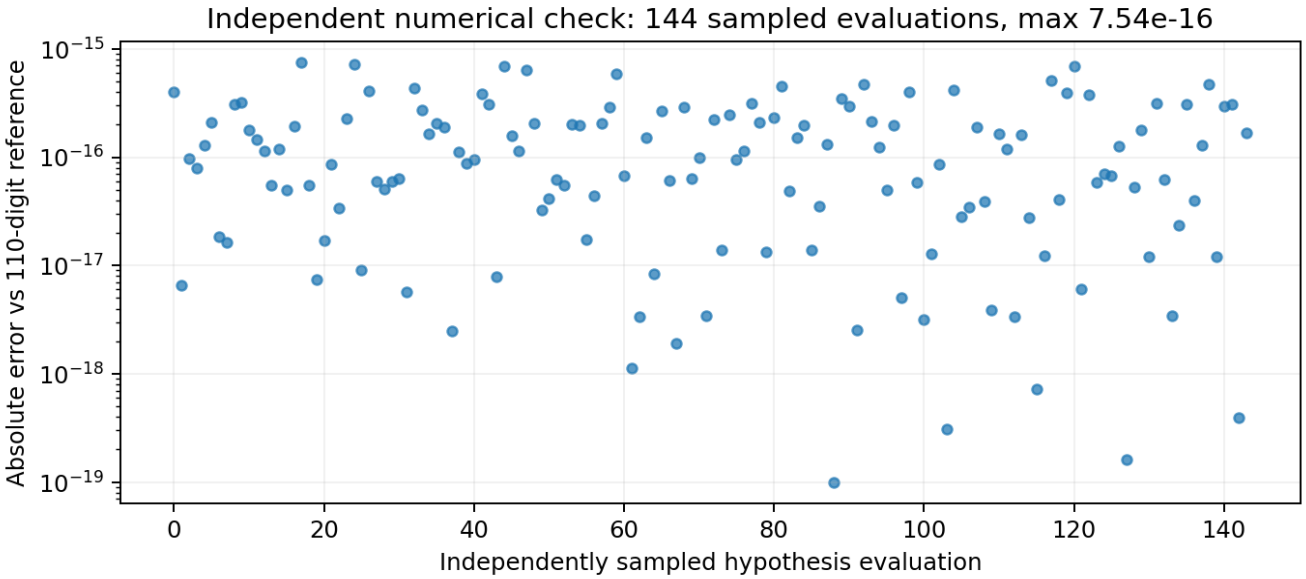


Figure 6. Observed absolute numerical differences to an independent 110-digit reference for the sampled graph outputs; the vertical axis is logarithmic.

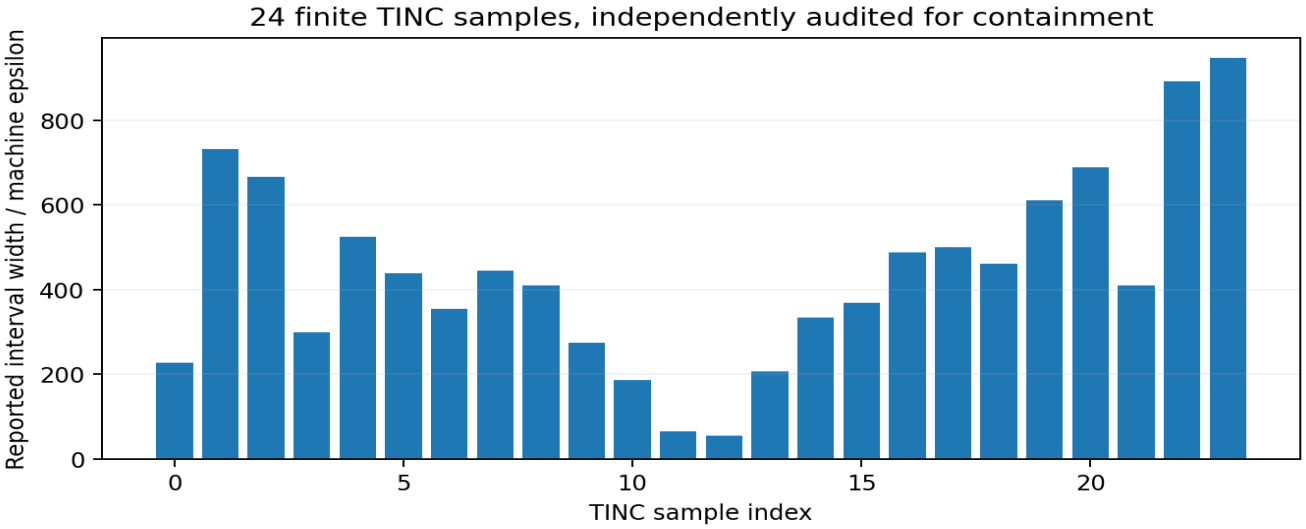


Figure 7. Reported interval widths in units of double-precision machine epsilon. The 24 sample-by-sample containment checks succeeded.

The high-precision reference uses the exact binary-double input values recorded in the CSVs. This avoids conflating decimal-text conversion error with the arithmetic executed by the engine. The successful sample audit is not a proof of correctness for all future candidate expressions.

8. Reproducibility, scope and scientific outlook

8.1 Verified computing environment

Debian GNU/Linux 13 (trixie), Linux x86_64; Intel Xeon Platinum 8573C, five visible logical CPUs; GCC 14.2 and Clang; C++17; official paceval API 4.25 forwarding files with the compatible original paceval Linux runtime. Release and AddressSanitizer/UndefinedBehaviorSanitizer builds were evaluated. These are environment descriptors; no speedup is inferred.

8.2 Repeatable build and audit

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release \
  -DPACEVAL_RUNTIME=/absolute/path/libpaceval_linux_sharedLIB.so
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/hypothesis_test --candidates 10000 --batch 256 --output reproduced
python3 src/high_precision_reference.py reproduced
```

The full 10,000-candidate study was also executed with batch sizes 256 and 37. Candidate, shortlist, prediction, numerical-audit and TINC CSV outputs were byte-identical. All three automated tests passed under both compilers and under sanitizers. The final publication also includes the full source and the frozen numerical outputs.

The candidate-count and batch-size limits exposed by this example program are host-level resource-policy choices, not intrinsic expression-length or variable-count limits of paceval.

8.3 Scientific interpretation and next steps

This study demonstrates a controlled evaluation layer for reproducibly generated mathematical hypotheses. Future work can ingest expressions from an external symbolic-regression or LLM-based generator, widen the functional grammar, incorporate noisy experimental datasets with documented uncertainty, compare independent model-selection methods, and analyze more demanding multidimensional inputs. A real scientific discovery claim would require externally supplied phenomena and independent domain-specific validation.

8.4 Source and reproducibility references

[1] Brunton, Proctor and Kutz (2016). Discovering governing equations by SINDy. doi.org/10.1073/pnas.1517384113.

[2] PySR / SymbolicRegression.jl. Searchable symbolic expressions, complexity selection and Pareto concepts. pysr.ai.

[3] COCO: benchmarking methodology and reproducibility. coco-platform.org.

[4] paceval API 4.25: computation graph creation and batched evaluations. paceval.com/api.

Publication evidence: src/hypothesis_test.cpp; src/high_precision_reference.py; examples/validation.json; examples/high_precision_audit.json; examples/hypotheses.csv; examples/dataset.csv. All are supplied alongside this report.